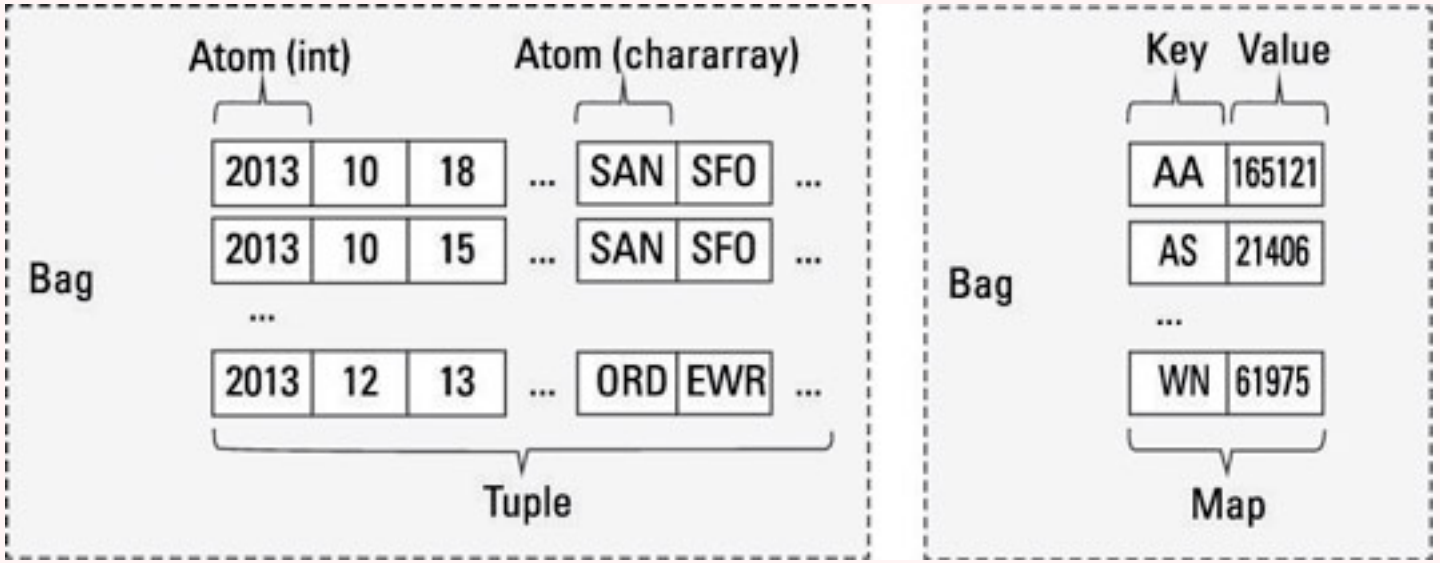
Analyse Data with PIG

Pig Data Types

- Field
- Complex
  - Tuple
  - Map
  - Bag (Relations)



Pig Fields

- Typed Values
- Int (signed 32 bit)
- Long (signed 64 bit)
- Float (32 bit fp)
- Double (64 bit fp)
- Chararray (utf-8 string)
- Bytearray (blob)
- Boolean

Pig Tuples and Maps

- A Tuple is an Ordered set of Fields
- Represented by parentheses
- Example
  - `('bob',55,12.2 )`

- Maps are key value pairs
- Represented as [key#value]
- Example
  - `['name'#'bob', 'age'#'55']`

Pig Bags

- A Bag is an unordered collection of Tuples
  - A Relation is a Bag of Bags
- Represented by { }
  - `{('bob',55,12.2 ), ('sally',52,11.3 )}`
- Tuples in the same bag can have different numbers of fields (remember no fixed schema!)

How to analyze data with PIG

- Load (…data into Pig)
- Transform (… data with operations)
- Dump/Save (… results somewhere)

https://pig.apache.org/docs/r0.11.1/start.html

LOAD

```
X = Load `data` USING PigStorage(‘,’) AS (type:fieldname,…)
```

Pre-existing function to handle commas

Assign a schema

- Beyond comma separated values, you can also load
  - HBASE tables
  - JSON files

DUMP & STORE

```
X = LOAD`data` USING PigStorage(‘,’) AS (type:fieldname,…);

DUMP X;
```

Results on standard output

```
STORE X INTO ‘output’ USING PigStorage(‘*’);
```

Allows for program/job optimization in batch mode

- Lazy evaluation: Nothing happens until a DUMP or STORE statement

Example 1

- We have a table

`urls: (url, category, pagerank)`

- Where
  - `url` is the url of a web page
  - `category` is a predefined category for that webpage
  - `pagerank` is the numericalvalue of the pagerank for that webpage

- Query:
  - *Find, for each sufficiently large category, the average pagerank of high-pagerank urls in that category*

Example 1 (SQL)

```
SELECT category, AVG(pagerank)
FROM urls WHERE pagerank > 0.2
GROUP BY category HAVING COUNT(*) > 10^6;
```

- Query:
  - *Find, for each sufficiently large category, the average pagerank of high-pagerank urls in that category*

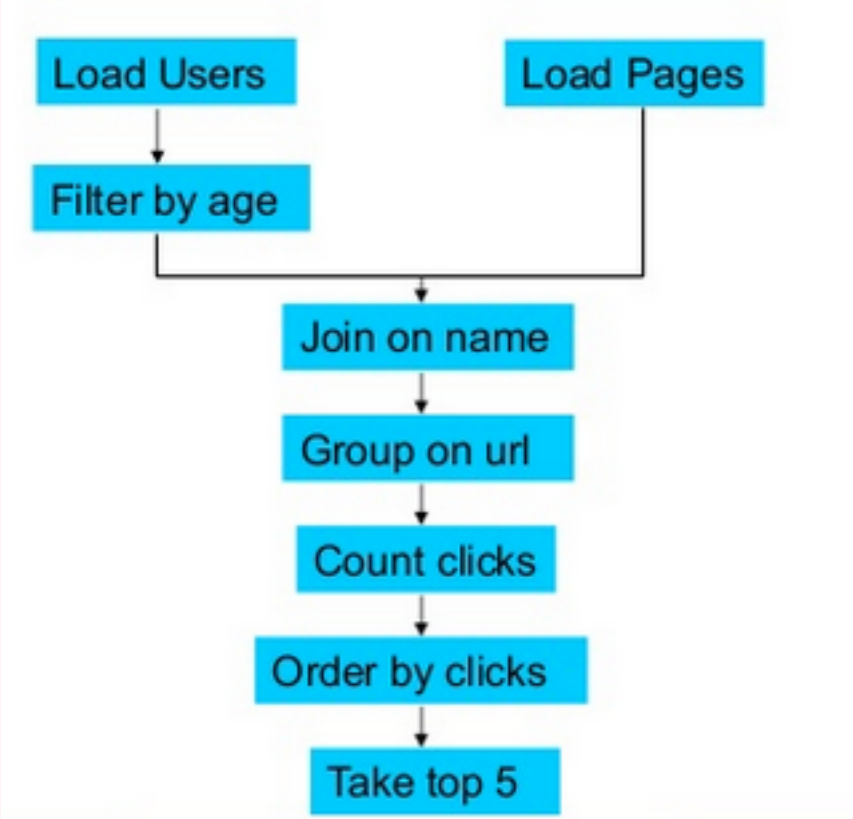
Example 1 (Pig Latin)

```
pgr_urls = FILTER urls BY pagerank > 0.2;
groups = GROUP pgr_urls BY category;
big_groups = FILTER groups BY COUNT(pgr_urls) > 10^6;
output = FOREACH big_groups GENERATE
         category, AVG(pgr_urls.pagerank);
```

- Query:
  - *Find, for each sufficiently large category, the average pagerank of high-pagerank urls in that category*

Example 2

- We have two files
  - User data
  - Website data
- Query:
  - Find top 5 most visited sites by users aged in the range (18,25)



Example 2 (MapReduce)

- Hundreds lines of code, ours to write

Example 2 (Pig Latin)

```
Users    = LOAD ‘users’ AS (name, age);
Filtered = FILTER Users BY age >= 18 and age <= 25;
Pages    = load ‘pages AS (user, URL);
Joined   = JOIN Filtered BY name, Pages BY user;
Grouped  = GROUP Joined BY url;
Sum = FOREACH Grouped GENERATE group, COUNT(Joined) AS clicks;
Sorted = ORDER Sum BY clicks DESC;
Top5   = LIMIT Sorted 5;
STORE Top5 INTO ‘top5sites’;
```

- Few lines of code, few minutes to write

Pig Execution Modes

- **Local**: single (Linux) machine, no need for Hadoop HDFS
  - pig –x local file.pig

- **MapReduce** (default): you need access to a Hadoop cluster and HDFS
  - pig –x mapreduce file.pig

Packaging Pigs

- **Scripts** (batch mode): a file.pig contains your Pig Latin statements and Pig commands

- **Grunt Shell** (interactive mode): Grunt interprets Pig Latin statements and Pig commands and you immediately see the output (good for prototyping)

- **Embedded**: execution within Java, Python or Javascript

UDFs

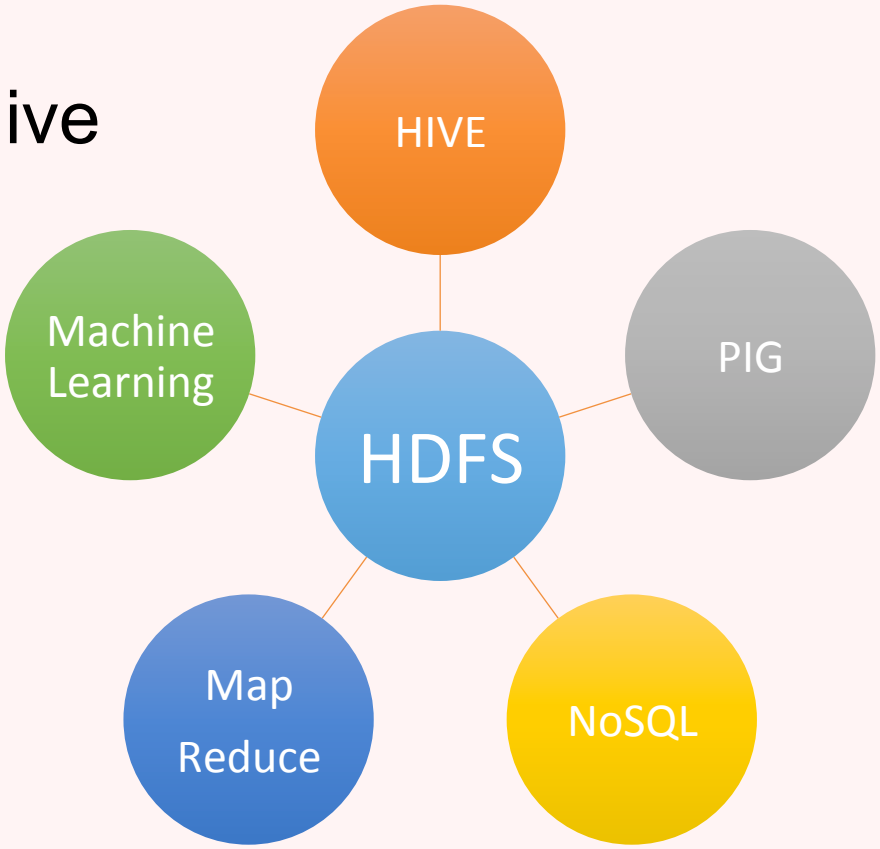
- PIG supports User-Defined Functions in different languages:
  - Python / Jython
  - Javascript
  - Ruby
  - Groovy
  - Java (Preferred)
- Ready function shared by the community in PiggyBank
- Different Types of Function
  - Eval UDFs (for GENERATE)
  - Aggregation UDFs (for GROUP)
  - Filter UDFs (for FILTER)
  - LOAD/STORE UDFs

Conclusion

- Pig is used to transform data
- More Like a Programming language than Hive

- MapReduce is the heart of this process
  - But it's file-based and can be high-latency

- Often have a data ecosystem
  - Different parts for different uses
  - Different response requirements and latency



End of Content

Apache PIG

Variations

- Filter based on string value

```
Y = FILTER X BY genres (matches '.*Comedy.*');
```

- Generate


```
Z = FOREACH X \
    GENERATE title, STRSPLIT(genres, "|") AS genres;
```

- GROUP

```
A = GROUP Z BY genres;
```

https://www.qubole.com/resources/cheatsheet/pig-function-cheat-sheet/